

AGENTS.md - Containers Module

INHERITED FROM the Helix Constitution

This module is governed by the Helix Constitution. All rules in the constitution^{™s} `AGENTS.md` and the `Constitution.md` it references apply unconditionally. Locate the constitution from any nested depth via its `find_constitution.sh` helper ^{“do NOT} hardcode a path (this module stays fully decoupled and project-agnostic per [“]§11.4.28).

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

Module Overview

`digital.vasic.containers` is a generic, reusable Go module for container orchestration, health checking, lifecycle management, and service discovery. It provides a unified interface for Docker, Podman, and Kubernetes runtimes with advanced features like lazy booting, idle shutdown, semaphore-based control, and resource monitoring.

Module path: `digital.vasic.containers` **Go version:** 1.24+ **Dependencies:** Container runtime clients (Docker/Podman/K8s), Prometheus client, minimal external dependencies

Package Responsibilities

Package	Path	Responsibility
runtime	<code>pkg/runtime/</code>	Container runtime abstraction layer: <code>ContainerRuntime</code> interface with implementations for Docker, Podman, Kubernetes. Auto-detection of available runtime. Runtime-agnostic container operations (start, stop, inspect, remove).
compose	<code>pkg/compose/</code>	Docker Compose orchestration: <code>ComposeOrchestrator</code> interface for compose file operations (up, down, restart). Support for profiles and service filtering. Multi-file composition and variable substitution.
health	<code>pkg/health/</code>	Health checking dispatcher: Multiple strategies (TCP, HTTP, gRPC, custom script). Retry policies with exponential backoff. Configurable timeouts and thresholds. Health status aggregation.
endpoint	<code>pkg/endpoint/</code>	Service endpoint configuration: <code>Endpoint</code> struct (host, port, path, scheme). Builder pattern for endpoint construction. Endpoint validation and URL generation.

Package	Path	Responsibility
lifecycle	pkg/ lifecycle/	Advanced lifecycle management: Lazy boot (on-demand container startup). Idle shutdown (resource optimization). Semaphore-based parallelism control. Graceful shutdown sequences.
monitor	pkg/monitor/	Resource monitoring: System resource tracking (CPU, memory, disk). Per-container resource usage. Threshold-based alerting. Prometheus metrics export.
event	pkg/event/	Lifecycle event bus: Publish/subscribe for container lifecycle events. Event types: ContainerStarted, ContainerStopped, HealthCheckFailed, etc. Hook system for custom actions.
discovery	pkg/ discovery/	Service discovery: TCP port scanning for service detection. DNS-based discovery. mDNS support for local network. Multi-strategy discovery with fallback.
logging	pkg/logging/	Logging abstraction: Bring-your-own-logger interface. Adapters for popular loggers (logrus, zap, zerolog). Structured logging support.
metrics	pkg/metrics/	Metrics collection: Prometheus-compatible metrics. Container lifecycle metrics. Health check metrics. Resource utilization metrics.
boot	pkg/boot/	High-level orchestration: BootManager composing all packages. One-line service initialization. Coordinated health checking and lifecycle management. Configuration validation. Distributor integration for remote endpoints.
orchestrator	pkg/ orchestrator/	Service orchestration: DefaultOrchestrator for auto-discovering and managing containerized services. Supports local and remote deployment. Auto-discovery of docker-compose files. Thread-safe service management.
remote	pkg/remote/	Remote host management: RemoteExecutor (SSH command execution with ControlMaster pooling), HostManager (host registry, resource probing), RemoteRuntime (ContainerRuntime over SSH), RemoteComposeOrchestrator.
scheduler	pkg/ scheduler/	Resource-aware container scheduling: 5 strategies (resource_aware, round_robin, affinity, spread, bin_pack). ResourceScorer for weighted host scoring (CPU 40%, Memory 40%, Disk 10%, Network 10%).
network	pkg/network/	Cross-host networking: TunnelManager for SSH tunnels (local/remote forwarding), PortAllocator (thread-safe port range

Package	Path	Responsibility
		20000-30000), OverlayNetwork for Docker overlay spanning hosts.
volume	pkg/volume/	Remote volume management: VolumeManager with 3 backends â€” SSHFS (real-time), NFS (shared export), rsync (periodic sync). Mount/unmount/sync operations.
envconfig	pkg/envconfig/	Environment configuration: CONTAINERS_REMOTE_* env var parsing, .env file loading, numbered host definitions (HOST_N_NAME/ADDRESS/PORT/...), template generation.
distribution	pkg/distribution/	Distribution orchestrator: Distributor composing scheduler + remote + network + volume. 7-phase workflow (probe â†’ schedule â†’ volumes â†’ deploy â†’ tunnels â†’ health â†’ events). Failover detection and rescheduling.
ctop	pkg/ctop/	Container monitoring: top/htop-style display for local and remote containers. Collector gathers container data. Display provides interactive TUI, snapshot, and JSON output. Sorting, filtering, color-coded resource usage.

Dependency Graph

```

boot ---> runtime
boot ---> compose ---> runtime
boot ---> health ---> endpoint
boot ---> lifecycle ---> runtime, event
boot ---> monitor ---> runtime, remote
boot ---> discovery ---> endpoint
boot ---> event
boot ---> logging
boot ---> metrics
boot ---> remote
boot ---> scheduler ---> remote

orchestrator ---> compose
orchestrator ---> remote
orchestrator ---> health
orchestrator ---> logging

distribution ---> scheduler ---> remote
distribution ---> remote
distribution ---> network ---> remote
distribution ---> volume ---> remote
distribution ---> runtime
distribution ---> logging

envconfig ---> remote

remote ---> runtime (RemoteRuntime implements ContainerRuntime)

```

```
remote ---> compose (RemoteComposeOrchestrator implements ComposeOrchestrator)
ctop ---> remote
ctop ---> envconfig
```

runtime, endpoint, and logging are leaf packages. boot, orchestrator, distribution, and ctop are integration layers. remote is the foundation for all distributed features.

Key Files

File	Purpose
pkg/runtime/runtime.go	ContainerRuntime interface and implementations
pkg/runtime/docker.go	Docker client implementation
pkg/runtime/podman.go	Podman client implementation
pkg/runtime/kubernetes.go	Kubernetes client implementation
pkg/runtime/autodetect.go	Runtime auto-detection logic
pkg/compose/compose.go	ComposeOrchestrator interface
pkg/compose/docker_compose.go	Docker Compose implementation
pkg/health/health.go	HealthChecker interface and dispatcher
pkg/health/tcp.go	TCP health check implementation
pkg/health/http.go	HTTP health check implementation
pkg/health/grpc.go	gRPC health check implementation
pkg/lifecycle/lifecycle.go	LifecycleManager interface
pkg/lifecycle/lazy_boot.go	Lazy boot implementation
pkg/lifecycle/idle_shutdown.go	Idle shutdown implementation
pkg/boot/manager.go	BootManager main orchestration logic
pkg/boot/options.go	BootManager functional options
pkg/orchestrator/orchestrator.go	ServiceOrchestrator for auto-discovery and management
pkg/orchestrator/orchestrator_test.go	Orchestrator unit tests
pkg/ctop/types.go	Ctop type definitions (ContainerProcess, DisplayConfig)
pkg/ctop/collector.go	Container data collection from local and remote hosts
pkg/ctop/display.go	Terminal UI display with sorting and filtering
cmd/ctop/main.go	Ctop CLI entry point
go.mod	Module definition and dependencies
CLAUDE.md	AI coding assistant instructions
README.md	User-facing documentation with quick start

Agent Coordination Guide

Division of Work

When multiple agents work on this module simultaneously, divide work by package boundary:

1. **Runtime Agent** â€œ Owns `pkg/runtime/`. Changes to runtime interface affect `compose`, `lifecycle`, and `monitor` packages. Must coordinate before modifying `ContainerRuntime` interface.
2. **Health Agent** â€œ Owns `pkg/health/`. New health check strategies can be added independently. Changes to `HealthChecker` interface require boot package updates.
3. **Lifecycle Agent** â€œ Owns `pkg/lifecycle/`. Complex lifecycle logic. Coordinates with runtime and event agents for state management.
4. **Boot Agent** â€œ Owns `pkg/boot/`. Integration layer. Requires testing against all package combinations.
5. **Discovery Agent** â€œ Owns `pkg/discovery/`. Independent service discovery logic. Can work in parallel with other agents.
6. **Monitor Agent** â€œ Owns `pkg/monitor/`. Resource tracking. Can work independently but coordinates with runtime for container metrics.
7. **Orchestrator Agent** â€œ Owns `pkg/orchestrator/`. Service orchestration with auto-discovery. Coordinates with `compose` and remote agents for deployment.
8. **Ctop Agent** â€œ Owns `pkg/ctop/`. Container monitoring with `top/htop`-style display. Coordinates with remote agent for multi-host collection. Independent display logic.

Coordination Rules

- **Runtime interface changes** require all agents to update. The `ContainerRuntime` interface is the shared contract.
- **Health checker** and **discovery** packages are independent and can be modified in parallel.
- **Boot package** integrates all packages. Any interface change in sub-packages requires corresponding boot updates.
- **Lifecycle** and **event** packages are tightly coupled. Coordinate changes to event types and lifecycle states.
- **Test isolation**: Each package has its own `_test.go` files. Boot tests import all packages for integration scenarios.
- **No circular dependencies**: The dependency graph is strictly acyclic. Never import `boot` from sub-packages.

Safe Parallel Changes

These changes can be made simultaneously without coordination: - Adding a new runtime implementation (e.g., LXC) to `pkg/runtime/` - Adding a new health check strategy to `pkg/health/` - Adding new discovery mechanisms to `pkg/discovery/` - Adding new event types to `pkg/event/` - Adding new scheduling strategies to `pkg/scheduler/` - Adding new volume backends to `pkg/volume/` - Adding new tests to any package - Updating documentation

Changes Requiring Coordination

- Modifying the `ContainerRuntime` interface (affects `remote.RemoteRuntime`)
- Changing `HealthChecker` interface signature
- Modifying `RemoteExecutor` interface (affects `scheduler`, `network`, `volume`, `distribution`)

- Modifying `HostManager` interface (affects scheduler, distribution, boot)
- Modifying `Scheduler` interface (affects distribution, boot)
- Modifying lifecycle state machine
- Adding new configuration fields to `boot.Config`
- Changing event types used across packages
- Modifying metrics schema

Remote Distribution Agents

1. **Remote Agent** â€œ Owns `pkg/remote/`. Foundation for all distributed features. Changes to `RemoteExecutor` or `HostManager` interfaces affect scheduler, network, volume, and distribution packages.
2. **Scheduler Agent** â€œ Owns `pkg/scheduler/`. Strategy implementations are independent. Changes to `Scheduler` interface require distribution and boot updates.
3. **Network Agent** â€œ Owns `pkg/network/`. Tunnel management and port allocation. Can work independently.
4. **Volume Agent** â€œ Owns `pkg/volume/`. Volume backend implementations (SSHFS/NFS/rsync) are independent.
5. **Distribution Agent** â€œ Owns `pkg/distribution/`. Top-level orchestrator. Requires testing against all remote packages.
6. **EnvConfig Agent** â€œ Owns `pkg/envconfig/`. Environment parsing. Independent of other packages except remote types.
7. **Ctop Agent** â€œ Owns `pkg/ctop/`. Container top monitoring. Uses `remote.HostManager` for remote collection. Independent display rendering. Can add new sorting/filtering without coordination.

Build and Test Commands

```
# Build all packages
go build ./...

# Run all tests with race detection
go test ./... -count=1 -race

# Run unit tests only (short mode)
go test ./... -short

# Run integration tests (requires Docker/Podman)
go test -tags=integration ./...

# Run benchmarks
go test -bench=. ./tests/benchmark/

# Run a specific test
go test -v -run TestBootManager_Start ./pkg/boot/

# Format code
gofmt -w .

# Vet code
go vet ./...
```

Commit Conventions

Follow Conventional Commits with package scope:

```
feat(runtime): add LXC runtime support
feat(health): add Redis health check strategy
feat(lifecycle): implement graceful shutdown with timeout
fix(boot): prevent race condition in parallel health checks
fix(compose): handle profile selection correctly
test(runtime): add Docker client integration tests
docs(containers): update API reference with lifecycle examples
refactor(health): extract retry logic to separate package
```

Thread Safety Notes

- **BootManager** is fully thread-safe. All public methods use `sync.RWMutex` for state protection.
- **Runtime implementations** use per-client locking for API calls.
- **HealthChecker** executes health checks concurrently but uses mutexes for result aggregation.
- **LifecycleManager** uses atomic operations for state transitions.
- **EventBus** (from `pkg/event/`) is thread-safe with internal locking.
- **MetricsCollector** uses `sync.Map` for concurrent metric updates.

Configuration Example

```
package main

import (
    "digital.vasic.containers/pkg/boot"
    "digital.vasic.containers/pkg/runtime"
    "digital.vasic.containers/pkg/logging"
)

func main() {
    // Auto-detect runtime
    rt, _ := runtime.AutoDetect()

    // Create logger
    logger := logging.NewDefaultLogger()

    // Create boot manager with functional options
    manager := boot.NewBootManager(
        boot.WithRuntime(rt),
        boot.WithLogger(logger),
        boot.WithHealthCheckRetries(3),
        boot.WithParallelStartup(true),
        boot.WithLazyBoot(true),
    )

    // Add services
    manager.AddService("postgresql", boot.ServiceConfig{
```

```

        ComposeFile: "docker-compose.yml",
        ServiceName: "postgres",
        HealthCheck: boot.TCPCheck("localhost", 5432),
        Required:    true,
    })

    // Start all services
    manager.Start(ctx)
}

```

Runtime Detection Logic

```

// AutoDetect tries: Docker -> Podman -> Kubernetes (in that order)
func AutoDetect() (ContainerRuntime, error) {
    // 1. Try Docker
    if dockerAvailable() {
        return NewDockerRuntime()
    }

    // 2. Try Podman
    if podmanAvailable() {
        return NewPodmanRuntime()
    }

    // 3. Try Kubernetes
    if kubernetesAvailable() {
        return NewKubernetesRuntime()
    }

    return nil, ErrNoRuntimeAvailable
}

```

Health Check Strategies

Strategy	Use Case	Configuration
TCP	Database, cache, message queue	Host, port, timeout
HTTP	REST APIs, web services	URL, expected status code, timeout
gRPC	gRPC services with health check protocol	Host, port, service name
Custom	Custom health logic	Script path or function

Lifecycle States

```

UNSTARTED -> STARTING -> STARTED -> STOPPING -> STOPPED
              |                               |
              +---> FAILED <-----+

```


Best Practices

1. Always Use Auto-Detection

```
// Good
runtime, err := runtime.AutoDetect()

// Bad
runtime := runtime.NewDockerRuntime() // Hardcoded
```

2. Configure Health Checks for All Services

```
// Good
manager.AddService("redis", boot.ServiceConfig{
    HealthCheck: boot.TCPCheck("localhost", 6379),
})

// Bad - no health check
manager.AddService("redis", boot.ServiceConfig{})
```

3. Mark Critical Services as Required

```
// Database is critical
manager.AddService("postgres", boot.ServiceConfig{
    Required: true, // Fail fast if unavailable
})

// Optional service
manager.AddService("optional-cache", boot.ServiceConfig{
    Required: false, // Continue if unavailable
})
```

4. Use Lazy Boot for Optional Services

```
manager := boot.NewBootManager(
    boot.WithLazyBoot(true), // Start services on-demand
)
```

5. Monitor Resource Usage

```
monitor := monitor.NewResourceMonitor(runtime)
metrics := monitor.GetContainerMetrics("postgres")
if metrics.MemoryPercent > 90.0 {
    logger.Warn("High memory usage detected")
}
```

Remote Distribution Configuration

Remote hosts are configured via environment variables or `.env` files. See `.env.example` for the full template.

```
# Enable remote distribution
CONTAINERS_REMOTE_ENABLED=true
CONTAINERS_REMOTE_SCHEDULER=resource_aware

# Define remote hosts (numbered 1, 2, 3, ...)
CONTAINERS_REMOTE_HOST_1_NAME=gpu-server-1
CONTAINERS_REMOTE_HOST_1_ADDRESS=192.168.1.100
CONTAINERS_REMOTE_HOST_1_RUNTIME=docker
CONTAINERS_REMOTE_HOST_1_LABELS=gpu=true,arch=amd64
```

Last Updated: February 22, 2026 **Version:** 2.1.0 **Status:** Production Ready